Did Bitcoin miners leave Bitcoin for AI?

Mini-projet Séries Temporelles - Tek-Up
Claude Code prompts for phases 2 to 9

How to use this document

- Phase 1 (architecture + data + README) is already done. This PDF holds the prompts for **every later phase**.
- Run **one phase per Claude Code session**. Long sessions lose context; short focused ones give better code.
- Start every session by pasting the **Session header** (next section), then the prompt of the phase.
- Before moving on, open the notebook, look at the figures and read the 5-line summary. If something looks wrong, fix it in the same session.
- **PROGRESS.md** is the project's memory: each phase updates it, and the next phase reads it first.
- Phases depend on each other: 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9. Do them in order.

Roadmap

Phase	Content	Brief section	Points
1 (done)	Architecture, data download, README	1	A
2	EDA, stationarity, pivot-date detection, split	1.3, 2	A (10)
3	ARIMA / SARIMA - Python + R	3.1	A
4	VAR, Granger, Johansen, VECM - Python + R	3.2	A + D
5	ARCH / GARCH - Python + R	3.3	B (4)
6	LSTM, GRU, walk-forward, hybrid	4	B + D
7	Forecast evaluation, Diebold-Mariano, rolling corr / DCC	6, 7	A + D
8	LLM material (you do the LLM part yourself)	5	C (4)
9	Report draft, defense notes, final check	8	all

Session header - paste this at the start of EVERY phase

It gives Claude Code the context and the rules again, because each new session starts from zero.

```
# Context
I'm continuing my time-series mini-project (Tek-Up, SDIA ING4):
"Did Bitcoin miners leave Bitcoin for AI?" (Option A - Finance).
AI converts: IREN, CIFR, WULF | Pure miners: MARA, CLSK, RIOT | References: BTC-USD, NVDA.

Before doing anything, read in this order:
1. docs/ the brief PDF (cahier des charges) - source of truth for requirements and grading
2. README.md and config file (tickers, dates, seed, split)
3. PROGRESS.md - what is already done, key results, open questions
4. data/DATA_DICTIONARY.md
Then tell me in 3 lines where the project stands before starting.

# Rules that always apply
- Load data ONLY from the saved snapshot in data/ (never re-download silently).
- Chronological 90/10 split from config. Fit scalers/transforms on train only.
- Fixed seeds. Relative paths only. Pinned package versions.
- Every code block: a one-line comment explaining WHY this choice, not what the code does.
- Every statistical test: state H0, the statistic, the p-value and a WRITTEN conclusion
  in plain words (French in markdown cells).
- Never invent numbers. Every number in text must come from executed output.
  Non-significant or failed results are reported honestly.
- Reuse helpers in src/ (Python) and R/ (R); add new helpers there, not copy-paste.
- Save every figure in figures/ (numbered, e.g. fig_03_02_acf_converts.png)
  and every result table as CSV in results/.
- Keep the code simple enough for me to explain it orally.

# End of session (mandatory)
1. Restart the kernel and run the notebook(s) of this phase top-to-bottom; fix every error.
2. Update PROGRESS.md: what was done, key results (with numbers), decisions taken and why,
   open questions, next phase.
3. Give me a 5-line summary of the findings.
4. Ask me before any decision that changes the research question or the asset list.
```

Phase 2 - EDA, stationarity and pivot date

Brief sections 1.3 and 2. Notebook: 02_eda.ipynb

Paste the Session header first, then:

```
# Phase 2 - Exploratory analysis (brief section 2)
Create notebooks/python/02_eda.ipynb.

1. Plots: log prices of the 6 miners, the CONVERTS and PURE indices vs BTC and NVDA
   (rebased to 100 at the start), log returns, 20-day realised volatility.
   Comment scales, trends and visible breaks.
2. ACF and PACF on the RAW series (log price) AND the DIFFERENCED series (log returns)
   for CONVERTS, PURE, BTC, NVDA. Write the interpretation of each (slow decay = non-
   stationarity, significant lags, what it suggests for p and q).
3. Decomposition: STL (period 5 = trading week; also test an annual period of 252).
   Comment trend, seasonal and remainder. If the seasonal component is negligible,
   quantify it (variance share) and say clearly that there is no real seasonality.
4. Outliers: rolling IQR (and/or Hampel filter) on returns. List the dates, link them
   to real events when obvious, and choose a justified treatment (keep / dummy variable /
   winsorise) - explain why. Save the outlier table to results/.
5. Stationarity: ADF AND KPSS on every series in level and in first difference.
   One summary table (statistic, p-value, conclusion). Conclude explicitly on the
   integration order d for each series (expected: I(1) prices, I(0) returns).
6. Pivot date: do NOT hard-code it from news. Detect it from data:
   - rolling 120-day correlation of CONVERTS with BTC and with NVDA,
   - rolling Engle-Granger cointegration p-value (CONVERTS vs BTC),
   - a structural break test (Chow at candidate dates, or Bai-Perron via the
     'ruptures' library / R 'strucchange') on the CONVERTS-BTC relationship.
   Propose ONE pivot date with justification and save it in the config file.
   Show me the evidence and wait for my OK before saving it.
7. Train/test split: apply the 90/10 chronological split, print the exact dates,
   and plot it. Check where the pivot falls relative to the test set.
```

Phase 3 - ARIMA / SARIMA (Python + R)

Brief section 3.1. Notebooks: 03_arima.ipynb + 03_arima.Rmd

Paste the Session header first, then:

```
# Phase 3 - ARIMA / SARIMA (brief section 3.1) - Python AND R
Create notebooks/python/03_arima.ipynb and notebooks/r/03_arima.Rmd
(same steps, same data snapshot, same split - results should match closely).

Target: the main series chosen in PROGRESS.md (CONVERTS index). Also run the
same pipeline on PURE for comparison.
1. Manual identification: from the ACF/PACF of phase 2, propose (p,d,q) and justify.
2. Automatic search: pmdarima.auto_arima (Python) / forecast::auto.arima (R).
   Compare manual vs automatic orders (AIC, BIC, parsimony) and pick one with reasons.
3. Seasonality: test whether a seasonal term is justified (weekly s=5). If not,
   explain why SARIMA is not used - with the evidence from phase 2.
4. Full residual diagnostics for the chosen model:
   Ljung-Box (several lags), Jarque-Bera (and Shapiro-Wilk), ARCH-LM,
   residual plots (time, ACF, histogram, QQ-plot). Write the conclusion of each test.
   If ARCH-LM rejects, say this motivates phase 5 (GARCH).
5. Forecast h = 5 and h = 10 on the test set with prediction intervals,
   plus a naive random-walk baseline. Save forecasts to results/forecasts_arima.csv
   in the common format: date, model, horizon, y_true, y_pred, lower, upper.
6. Short written comparison Python vs R (same orders? same coefficients?).
```

Phase 4 - VAR, Granger, Johansen, VECM (Python + R)

Brief section 3.2 - the core of the research question. Notebooks: 04_var_vecm.ipynb + 04_var_vecm.Rmd

Paste the Session header first, then:

```
# Phase 4 - Multivariate models (brief section 3.2) - Python AND R
Create notebooks/python/04_var_vecm.ipynb and notebooks/r/04_var_vecm.Rmd
(R: packages vars and urca).

This phase answers the research question. Do every step on the FULL sample
AND separately on PRE-pivot and POST-pivot sub-samples (pivot date from config).

1. VAR on log returns [BTC, NVDA, CONVERTS, PURE]:
   lag order by AIC/BIC/HQ (show the table, justify the choice), stability check
   (roots inside the unit circle), residual diagnostics (Portmanteau, normality, ARCH).
2. Granger causality (write H0 each time):
   BTC -> CONVERTS, BTC -> PURE, NVDA -> CONVERTS, NVDA -> PURE, and the reverse.
   One table pre vs post pivot. Interpret: did the "driver" of the converts change?
3. Impulse response functions (BTC shock and NVDA shock on each group), pre vs post.
4. Cointegration on LOG PRICES (all I(1) - confirmed in phase 2):
   Johansen (trace and max-eigen, justify the deterministic term choice) for
   (CONVERTS, BTC), (PURE, BTC), (CONVERTS, NVDA), (PURE, NVDA), pre and post pivot.
   Also Engle-Granger for each pair as a robustness check.
   Summary table: pair, period, rank, p-value / critical values, conclusion.
5. VECM wherever cointegration is found: lag order, cointegrating vector,
   adjustment coefficients (alpha). Interpret alpha as the speed of return to
   equilibrium, convert it to a half-life in trading days: ln(0.5)/ln(1+alpha).
6. Write the key finding in plain words: did the converts lose their leash with
   Bitcoin and gain one with NVDA? Is the pure-miner group a valid control?
   If the evidence is weak or mixed, say so - do not force the story.
7. Forecast h = 5 and 10 from the VAR (and VECM if fitted) for CONVERTS,
   saved in the common forecast format.
```

Phase 5 - ARCH / GARCH (Python + R)

Brief section 3.3. Notebooks: 05_garch.ipynb + 05_garch.Rmd

Paste the Session header first, then:

```
# Phase 5 - Conditional volatility (brief section 3.3) - Python AND R
Create notebooks/python/05_garch.ipynb (library arch) and notebooks/r/05_garch.Rmd (rugarch).

1. ARCH-LM test on the residuals of the best ARIMA (phase 3) and of the VAR (phase 4).
   Conclude on ARCH effects.
2. Fit GARCH(1,1) on CONVERTS returns (mean = the ARIMA from phase 3 or constant,
   justify). Compare with GARCH(1,1) Student-t, EGARCH(1,1), GJR-GARCH(1,1),
   and GARCH(2,1) only if diagnostics justify it. Comparison table: AIC, BIC,
   log-likelihood, alpha+beta (persistence), half-life of volatility shocks.
3. Diagnostics on STANDARDISED residuals and their squares:
   Ljung-Box, ARCH-LM, QQ-plot vs the assumed distribution. Show that the
   heteroscedasticity is gone (or say honestly if it is not).
4. Plot the conditional volatility over time for CONVERTS, PURE and BTC on one chart,
   annotate the pivot date and 4-6 known events (search the dates in the news and
   cite the source in a markdown cell; do not guess dates).
5. Does the volatility of the converts behave more like BTC or like NVDA after
   the pivot? Compare persistence parameters pre vs post pivot.
6. Volatility forecasts h = 5 and 10, saved to results/.
```

Phase 6 - LSTM, GRU, walk-forward, hybrid

Brief section 4 (Python only, labelled as such). Notebook: 06_deep_learning.ipynb

Paste the Session header first, then:

```
# Phase 6 - Deep learning (brief section 4) - Python only (state it at the top)
Create notebooks/python/06_deep_learning.ipynb. Use PyTorch or Keras (pick one,
pin the version, fix all seeds, set deterministic flags).

1. Data shaping: sliding windows on CONVERTS (same target as ARIMA) with optional
   exogenous inputs (BTC and NVDA returns, realised volatility). MinMax or Standard
   scaling fitted on TRAIN ONLY. Explain the window size choice (test 10, 20, 40).
2. Models: LSTM and GRU with the same budget (1-2 layers, dropout, early stopping).
   Justify each hyperparameter in one line. Keep a small, explicit grid - no huge search.
3. Training curves: train vs validation loss for each model, comment over/under-fitting.
4. Walk-forward validation (expanding window, e.g. 5 folds, TimeSeriesSplit style -
   NEVER k-fold). Report mean and std of RMSE/MAE across folds.
5. Hybrid (bonus, brief 4.2): ARIMA from phase 3 for the linear part + LSTM on the
   ARIMA residuals; final forecast = ARIMA + LSTM residual forecast.
6. Optional only if time allows: Prophet as an extra baseline (brief 4.3).
7. Forecasts h = 5 and 10 on the test set in the common forecast format
   (direct or recursive multi-step - say which and why).
8. Honest comparison with the random walk: if deep learning does not beat it,
   say it and explain why this is common for financial returns.
```

Phase 7 - Evaluation, Diebold-Mariano, extensions

Brief sections 6 and 7. Notebook: 07_evaluation.ipynb

Paste the Session header first, then:

```
# Phase 7 - Forecast evaluation (brief section 6) + extensions (section 7)
Create notebooks/python/07_evaluation.ipynb. It only READS results/forecasts_*.csv.

1. ONE summary table for all models (random walk, ARIMA, VAR, VECM, LSTM, GRU,
   hybrid, Prophet if any), for h = 5 and h = 10: MSE, RMSE, MAE, MAPE.
   MAPE on the price level (not on returns) - explain why in a note.
   Add columns: interpretability (low/medium/high) and training time.
   Save as results/model_comparison.csv and a nicely formatted figure.
2. Diebold-Mariano test (with Harvey-Leybourne-Newbold small-sample correction;
   implement it in src/metrics.py with a docstring and a unit test) for each model
   vs the random walk, and for the best classical vs best deep model.
   Table: DM statistic, p-value, conclusion.
3. Plot forecasts vs actual values with intervals for the 3 best models.
4. Conclusion: best model for this use case considering accuracy, significance,
   interpretability and compute cost - written in French.
5. Extensions (bonus): rolling 90-day correlation of CONVERTS and PURE with BTC
   and with NVDA on ONE chart, pivot date marked - this is the key figure of the
   project ("the crossing lines"). Then a DCC-GARCH (R rmgarch, or Python
   implementation if reliable) between CONVERTS and BTC, and CONVERTS and NVDA.
   If DCC-GARCH is too unstable, say so and keep the rolling correlation.
```

Phase 8 - LLM material (you do the LLM part yourself)

Brief section 5 (4 points). Folder: llm/

Paste the Session header first, then:

```
# Phase 8 - Prepare the LLM section (brief section 5)
IMPORTANT: do NOT write any LLM answer, any fake conversation, or any fake
"human vs LLM" comparison. I will do that part myself with a real LLM.

Create in llm/:
1. stats_summary.md - generated by code from results/: mean, variance, skewness,
   kurtosis of each series, ADF/KPSS table, chosen ARIMA, VAR/Granger, Johansen
   and GARCH results, model comparison table. Numbers only, no interpretation.
2. prompts.md - 4 ready-to-paste prompts (each tells the LLM to answer in French):
   a) "From these statistics, which models should I test and why?"
   b) "Explain these results to a non-statistician investment committee."
   c) "Give simulated investment recommendations from these forecasts, and list
      yourself all their limits and risks."
   d) A follow-up prompt asking the LLM to check its own statistical claims.
3. my_interpretation_TEMPLATE.md - sections I fill in BEFORE asking the LLM.
4. comparison_TEMPLATE.md - table: LLM claim | correct? | type of error
   (hallucinated number, invented causality, oversimplification, outdated fact) |
   my correction | evidence (figure/table reference).
5. A checklist of typical LLM errors to look for on THIS project
   (e.g. "miners are Bitcoin proxies", Granger causality = true causality,
   invented p-values, ignoring transaction costs).
```

Phase 9 - Report, defense notes, final check

Brief section 8. Folder: report/

Paste the Session header first, then:

```
# Phase 9 - Report draft and final check (brief section 8)
Prerequisite: I have filled in llm/ (my interpretation + real LLM answers + comparison).

1. Report draft in French (report/rapport.docx and PDF), 15-25 pages excluding
   annexes, following the brief order: introduction (theme, series, research question,
   one-sentence justification of the theme), data, EDA, classical models, GARCH,
   deep learning, forecast evaluation, LLM section (from MY files in llm/),
   critical discussion (section 8.4: model limits + general reflection on generative
   AI for time series in finance), conclusion. Annex: LLM prompts and full answers.
   Use the figures from figures/ and tables from results/ only.
   Mark every interpretation paragraph [A RELIRE] so I review it.
2. DEFENSE_NOTES.md: for each section - what was done, why, how to read the key
   figure/table, and 3 likely examiner questions with short answers
   (e.g. why ADF AND KPSS? why no k-fold? what does alpha in the VECM mean?
   why Diebold-Mariano and not just RMSE?).
3. Final reproducibility check: fresh virtual environment from requirements.txt,
   run every Python notebook and knit every Rmd top-to-bottom. List any failure.
4. Requirements checklist: every bullet of the brief (sections 1 to 8) with
   status done / partial / missing and the file where it is.
5. Build the submission archive: code + notebooks + report + data snapshot (zip).
```

What YOU must do yourself

The brief says every student must be able to explain and defend all of the work, including the code. These points can't be delegated:

- **After each phase:** read the notebook, look at every figure, and check that the written conclusions match the numbers.
- **Phase 2:** validate the pivot date yourself. It drives the whole analysis.
- **Phase 8 (4 points):** write your own interpretation **before** asking the LLM, run the prompts in a real LLM, copy the answers in full, and fill in the comparison table. The critique of the LLM must be yours.
- **Phase 9:** review every [A RELIRE] paragraph and rewrite it in your own words.
- **Before submitting:** read DEFENSE_NOTES.md and practise answering the questions out loud.
- Check the submission deadline with your instructor (the brief says 12 October 2026).

If a session goes wrong

- Claude Code forgot the context: paste the Session header again and ask it to re-read PROGRESS.md.
- A result contradicts the hypothesis: keep it. An honest negative result scores better than a forced story.
- A notebook breaks after a later phase: ask "Re-run notebooks 01 to N top-to-bottom and fix every error without changing results unless needed - tell me what changed."